

Estándar de codificación para Sistema de Prácticas Profesionales



Emiliano Morales Baizabal

PRINCIPIOS DE CONSTRUCCIÓN DE SOFTWARE –
FACULTAD DE ESTADÍSTICA E INFORMÁTICA
UNIVERSIDAD VERACRUZANA

Propósito	Establecer reglas y convenciones comunes para escribir código de manera consistente y así facilitar la legibilidad y mantenibilidad.
Idioma	Código completamente en español a excepción de palabras reservadas y prefijos de nombrado
Encabezados de archivos o clases	Comience cada programa con un encabezado descriptivo
Formato de encabezado	<pre>/* * Autor: nombre y primer apellido * Fecha de creación: dd/mm/aaaa * Descripción: Explicación breve y concisa del propósito de la clase (no explica su funcionamiento) */</pre>
Imports	No utilizar importaciones con comodín (Wildcard Imports)
Buen Import	<pre>import java.util.List; import java.util.ArrayList; import java.util.Map;</pre>
Mal import	<pre>Import java.util.*;</pre>
“Naming”	Usar nombres descriptivos para todas las variables, nombres de funciones, constantes, y otros identificadores. Usar sustantivos para nombres de clases. Usar adjetivos para nombres de interfaces. Usar verbos para nombres de métodos. Uso de variables de una sola letra solo permitidos en ciclos. Evitar uso de abreviaciones
Buen Ejemplo de “Naming”	<pre>int numeroEstudiantes;</pre>
Mal Ejemplo de “Naming”	<pre>int n;</pre>

Comentarios	• Documente situaciones inusuales en el código para que el lector pueda entender su operación. • Los comentarios deben explicar el propósito del código en lugar de describir de manera evidente las acciones que este realiza o repetir el código. • Comentarios al mismo nivel de indentación que el código relacionado. • Comentarios de una línea con el estilo <code>//...</code> • Comentarios multilínea con el estilo <code>/*...*/</code>
-------------	--

Buen Comentario	<pre>/* Se agrega 1 día adicional debido a que el sistema externo * maneja fechas con zona horaria UTC, lo que puede provocar * desfases al convertir a hora local */ fechaEntrega = fechaEntrega.plusDays(1);</pre>
Mal Comentario	<pre>/* Sumamos 1 día a la fecha de entrega */ fechaEntrega = fechaEntrega.plusDays(1);</pre>
Espacios en blanco	• Escribir los programas con suficiente espacio para que no se vean amontonados. • Separa secciones lógicas de código con una línea en blanco. • Incluir un espacio en blanco después de las palabras clave de control de flujo (if, for, while) • Incluir un espacio en blanco entre operadores para mejorar la legibilidad del código. • Incluir una línea en blanco entre la declaración de los atributos de una clase y sus métodos.
Indentación	• Indentar cada nivel de bloque de código respecto al anterior utilizando dos espacios. • No utilizar tabulaciones para la indentación. • Las llaves de apertura deben colocarse en la misma línea que la declaración de la estructura de control o bloque, mientras que la llave de cierre debe ubicarse en una nueva línea alineada con la estructura correspondiente, incluso en bloques no vacíos y estructuras similares a bloques. • Las llaves deben utilizarse en las sentencias de control de flujo, aun cuando el cuerpo esté vacío o contenga una sola sentencia.
Ejemplo de indentación	<pre>while (opcion != 99) { if (opcion > 0) { procesarOpcion(opcion); } else { mostrarError(); } }</pre>
Límite de Línea	Máximo 100 caracteres.
Estructura del Código	En una clase: 1. Constantes 2. Atributos 3. Constructores 4. Getters/Setters 5. Métodos Públicos 6. Métodos Privados

Manejo de errores	• No ignorar excepciones • Uso de “throws” •
Capitalization (Nombres de clases)	PascalCase
Capitalization (Nombres de interfaces)	PascalCase
Capitalization (Nombres de métodos)	camelCase
Capitalization (Nombres de variables)	camelCase
Capitalization (Nombres de constantes)	UPPER_SNAKE_CASE.
Capitalization (Nombres de paquetes)	En minúsculas y con nombre de dominio en reversa separado por puntos.
Ejemplo de “capitalization”	<pre>package uv.lis.inventario.tienda; public class ControladorInventario { private int cantidadProductos; protected String nombreProducto; public static final int LIMITE_MAXIMO = 100; public void actualizarInventario() { // implementación } } public interface Almacenable { void guardarProducto(); }</pre>
Java FX	
Java FX (cambia)	• Seguir las convenciones estándar de Java. • Los nombres de los controladores deben finalizar con la palabra “Controller”. • Implementar el patrón Modelo-Vista-Controlador (MVC) utilizando FXML y Java. • Los archivos FXML y sus controladores deben tener nombres relacionados que permitan identificar fácilmente que pertenecen a la misma vista.
Vistas	• Formato FXML (.fxml) • Nombrado con PascalCase

Ejemplo Nombre Vista	GUI-RegistroUsuario
Controladores	• Formato clase de Java (.java) • Nombrado con PascalCase • Nombrado con: Nombre de la vista + “Controller”
Ejemplo Nombre Controlador	GUI-RegistroUsuarioController
Uso de imágenes	• Formato .png, .jpg o .jpeg • Nombrado con prefijo “img” + PascalCase
Prefijos de nombrado de componentes	• Button: btn • Label: lbl • TextField: tf • PasswordField: pf • TextArea: ta • CheckBox: chk • RadioButton: rb • ComboBox: cmb • ListView: lv • TableView: tv • ProgressBar: pb • Slider: sld • Stage: stage • Scene: scene
Ejemplo de nombrado de componentes	<pre> Button btnGuardar; Label lblTitulo; TextField tfNombre; PasswordField pfContrasena; TextArea taDescripcion; CheckBox chkActivo; RadioButton rbMasculino; ComboBox<String> cmbOpciones; ListView<String> lvItems; TableView<Usuario> tvUsuarios; ProgressBar pbCarga; Slider sldVolumen; Stage stagePrincipal; Scene sceneLogin; </pre>

Bibliografía:

- Google. (s.f.). *Google Java Style Guide*.
<https://google.github.io/styleguide/javaguide.html#s4.4-column-limit>
- Oracle. (1999). *Code Conventions for the Java Programming Language*.
<https://www.oracle.com/java/technologies/javase/codeconventions-namingconventions.html>
- Oracle. (2013). JavaFX Best Practices. https://docs.oracle.com/javafx/2/best_practices/jfxpub-best_practices.htm

